A Look into the NVIDIA GPU Architecture

Contents

1 Introduction 2

2 Graphics Processing Unit 2

3 Architectural Conflict 2

4 GPU Hierarchy 3

5 The Volta Architecture 3

6 A Look into the Hierarchy 4

7 VRAM 5

8 Memory Hierarchy 5

9 A Segue into the Comparison 6

10 Kernel 7

11 Thread 7

12 Thread Block 7

13 Grid 8

14 Warp 9

15 A Fast Look into the CPU Architecture 10

16 Throughput and Latency 11

17 GPU and CPU Comparison 12

18 Source Material 12

1 Introduction

This paper is being written to help me, the team members, and outsiders (“hello outsider”) that are looking to set a foot into the GPU architecture and understand the difference between the CPU and GPU. The paper is written by Smai Abderrahman solely from the source materials and the courses issued by Dr. Hocine SAADI, at the time of me writing this sentence, I am a 3rd year bachelor student at the university of science and technology houari boumedien and this paper will contribute in the research of bachelor’s graduation project.

This paper is not meant to be highly accurate. It is meant to be a segue to gaining a general knowledge of the GPU architecture.

2 Graphics Processing Unit

A Graphics Processing Unit is a processor designed to perform a very large number of simple calculations at the same time. It was originally built to draw images on a screen, but today it is also used for any task that can be split into many small, independent pieces of work.

3 Architectural Conflict

With each period of time, a new NVIDIA architecture comes to the surface with new instructions, new components such as Tensor cores, and support for newer API versions: CUDA, Vulkan, OpenGL, OpenCL, . . . We will not get into the details of each architecture. We will get into the GPU hierarchy.

4 GPU Hierarchy

At a higher level, the GPU contains many Streaming Multiprocessors (SM), each of which contains many cores (generally a multiple of 32).

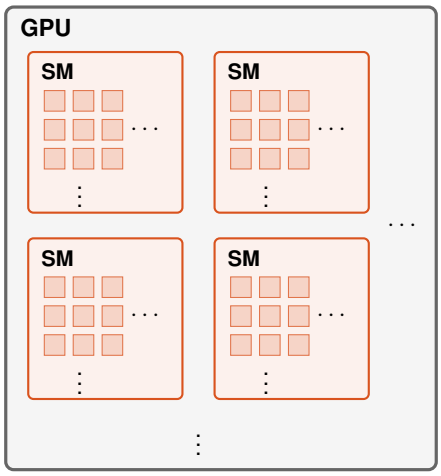


Figure 1. High-level GPU hierarchy.

Now, at a bit lower level, the organization, count, dependencies, and whole components differ! In the next section, I will give an example with the Volta architecture!

5 The Volta Architecture

GPU. Each Volta GPU contains many Streaming Multiprocessors (SM) and an L2 memory cache that is shared between all of the SMs.

Streaming Multiprocessor. Each SM holds four Processing Partitions, an L1 memory cache, and an L1 instruction cache. Both L1 caches are shared between the four Processing Partitions of that SM.

Processing Partition. Each Processing Partition, also called a Processing Block, contains an L0 instruction cache, a warp scheduler, a dispatch unit, and many registers called the Register File. It also contains a set of cores: FP32 and FP64 cores for floating-point operations, INT32 cores for integer operations, and Tensor cores for matrices and other similar operations, it's a super core really!

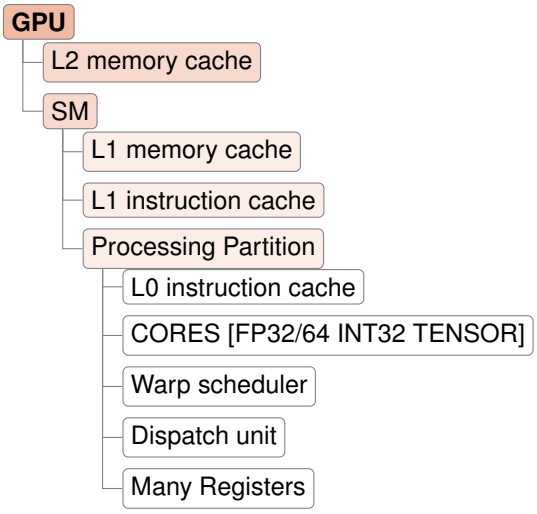


Figure 2. Volta GPU hierarchy.

> Other components were ignored for the sake of simplification.

Let's do some math on the Titan V, which has the GV100 GPU, with 80 SMs and 5120 FP32 cores in total. Dividing the cores between the SMs gives

$$\frac{5120 \text{ FP32 cores}}{80 \text{ SMs}} = 64 \text{ FP32 cores per SM.}$$

Since each SM has four Processing Partitions, every partition gets

$$\frac{64 \text{ FP32 cores}}{4 \text{ partitions}} = 16 \text{ FP32 cores per Processing Partition.}$$

6 A Look into the Hierarchy

At the leaves of the tree we find the following components. We will take a high-level look at each of them, and I will ignore some aspects for the sake of simplicity.

Cores, a core is an execution unit (an Arithmetic and Logic Unit, ALU), and each core type has a specific niche, e.g. FP cores for floating-point instructions, INT cores for integer instructions, Tensor cores for matrix operations, ...

Register File, holds the data of the active threads, such as operands, results, and local variables, used by the cores while executing, for near-zero latency during execution!

L0 instruction cache, holds the next instructions to be executed by the Processing Partition.

Warp scheduler, tracks the active warps (a warp is a group of 32 threads that execute the same instruction together), and decides which warp's next instruction is to be executed on the aforementioned cores.

> We will get deeper into what a warp and a thread are later on.

Dispatch unit, gets issued an instruction by the warp scheduler and routes it to the core pipeline required for that instruction, e.g. FP32/64, INT32, . . . , so the cores can execute it!

L1 instruction cache, holds the instructions of the warps, to be issued to the L0 instruction caches of the Processing Partitions.

L1 memory cache, holds the data needed by the warps to be executed, which is then loaded into the register file.

Processing Partition, is responsible for orchestrating the execution of multiple warps, using its own warp scheduler, dispatch unit, register file, and cores.

Streaming Multiprocessor, is responsible for executing the given threads in parallel, using its own Processing Partitions and the L1 caches shared between them.

L2 memory cache, sits outside of the SMs and is shared between all of them, it holds the data being processed, such as vertex buffer object data, textures, . . .

7 VRAM

You might have noticed that we have not yet mentioned the VRAM, also known as the DRAM in the GPU field (HBM2 on the V100), and there is a good reason for that: **speed**. The GPU requires speed for low latency, and the VRAM is quite slow compared to the aforementioned components. The VRAM holds the data the GPU might need later on, while the data being worked on is copied to the internal caches. It is connected to the GPU's memory controller, and we need it for the next section, the memory hierarchy.

8 Memory Hierarchy

In this section we will discuss the hierarchy of the GPU's memory and how the data gets from the VRAM to the Register File to be executed. You will also notice something interesting that justifies the need for the whole memory pipeline!

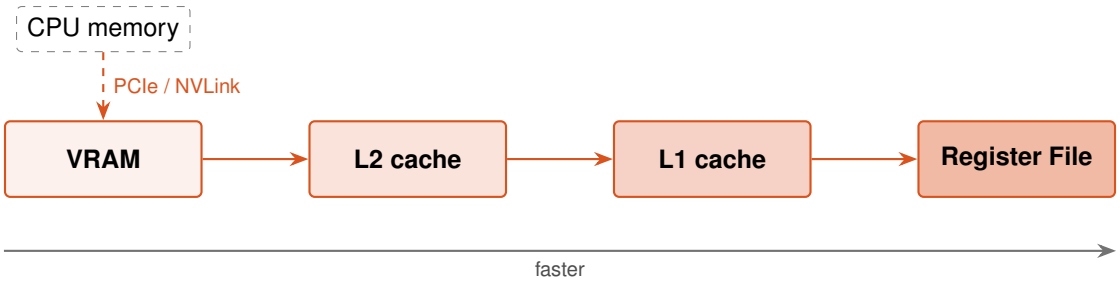


Figure 3. The path of the data from the CPU's memory to the Register File.

VRAM, the data is first copied from the CPU's main memory to the VRAM via PCIe or NVLink. This is relatively fast, though still much slower than reading from the VRAM itself.

L2 memory cache, the active data requested by the warps is copied from the VRAM, to be shared by all of the Streaming Multiprocessors. This memory has a very high speed.

L1 memory cache, the active data requested by the warps of the Processing Partitions of an SM is copied from the L2 memory cache. This memory runs at a much higher speed.

Register File, the active data needed by the cores is finally copied from the L1 memory cache, to be processed by the cores. This is the fastest memory!

Now, you might have noticed how interesting this pipeline is: the lower the hierarchy, the higher the speed. The whole data is always present on the VRAM until it is freed by the CPU. Assume a Vertex Buffer Object's data is to be rendered: when the draw order is requested, the vertices to be rendered first are copied to the L2 cache, then they get distributed between all of the L1 caches that require them, then the vertices required by the cores are copied to the Register File, which is reported to take negligible or near-zero clock cycles while the data is present in it.

9 A Segue into the Comparison

In reality, we need to understand kernels, threads, thread blocks, grids, and warps, and how they contribute to making the GPU good at parallelism. We will start with the kernel, since a thread is an instance of it. So before we dive into the comparison, I would like to get into all of them, and then get back to the comparison.

10 Kernel

A kernel is a function written to be executed on the GPU, and it is ordered to be executed from the CPU side of the machine!

11 Thread

A thread is an instance of the kernel.

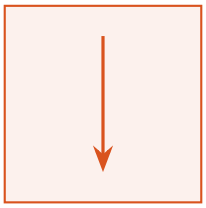


Figure 4. Thread icon.

12 Thread Block

A thread block is a group of threads that run in parallel with one another. Blocks are completely independent of each other in terms of execution. A block runs entirely on a single Streaming Multiprocessor (an SM can hold several blocks at once), and its threads are executed on the Processing Partitions of that SM in parallel.

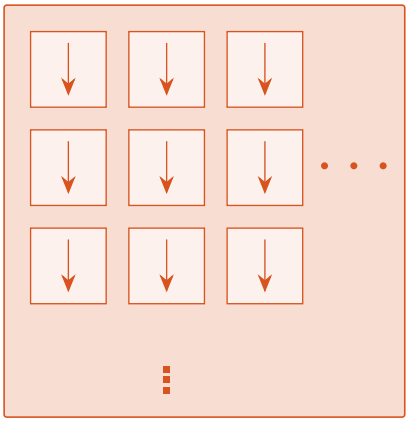


Figure 5. Thread block icon: a container of threads.

Let's take an example! A block of 64 threads runs on a GPU with 2 SMs, each with 4 Processing Partitions of 16 FP32 cores.

GPU: $2 \times 4 \times 16 = 128$ FP32 cores
Block: $64/32 = 2$ warps, on 1 SM and 2 partitions
Working: $2 \times 16 = 32$ cores

Most of the GPU sits idle!

Now let's split the same 64 threads into 2 blocks of 32 threads each!

Blocks: $32/32 = 1$ warp each, and since blocks are independent, each one can go to its own SM
1 partition on each SM is working

The second SM is no longer idle, it's doing work for us, and apparently the GPU loves to do stuff on its SMs, it does not like to stay idle!

13 Grid

A grid is all the thread blocks contributing to the execution of a kernel. By default, grids run sequentially, one after the other!

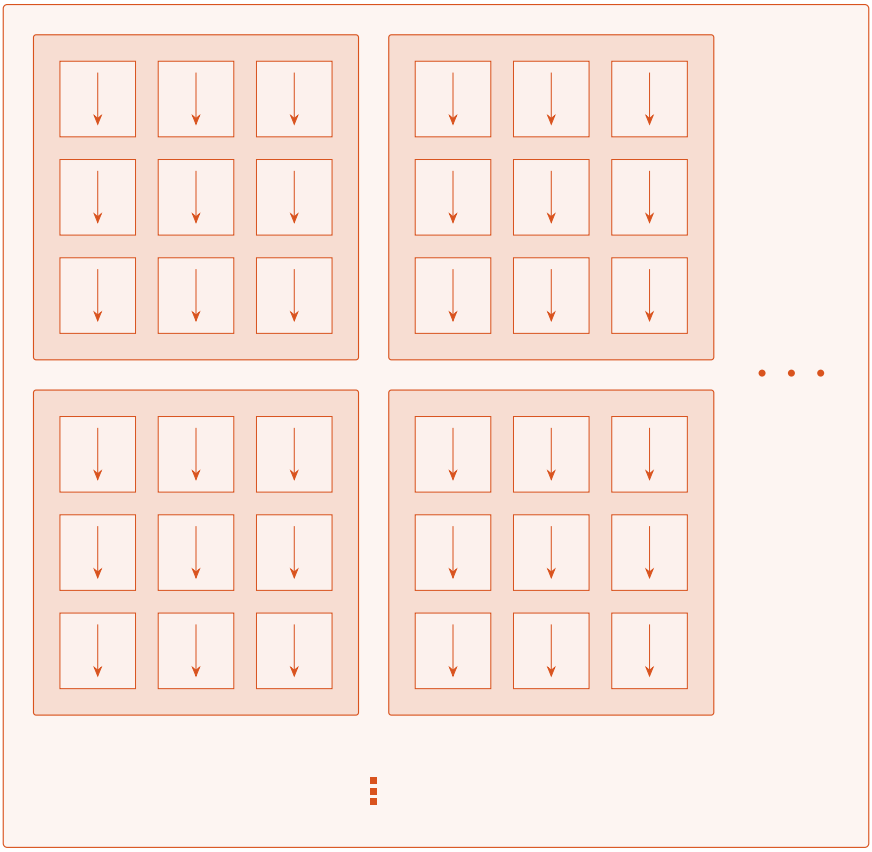


Figure 6. Grid icon: a container of thread blocks.

14 Warp

A warp is the unit of execution, it is a group of 32 threads within a block. Each warp is scheduled by a warp scheduler, so the Processing Partitions can process different warps in parallel.

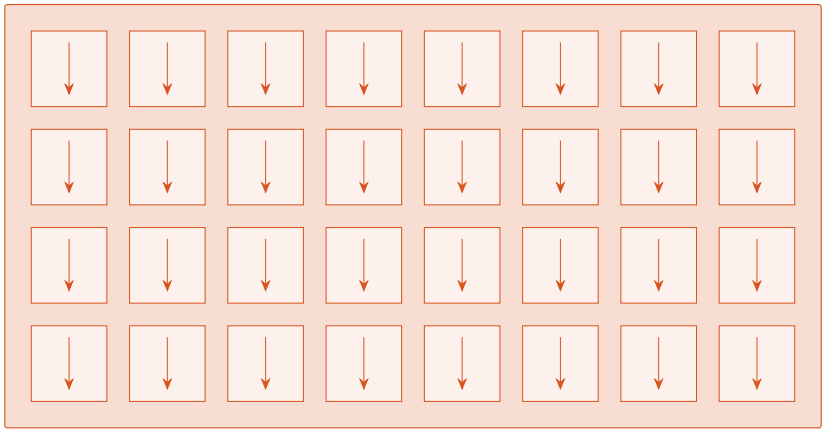


Figure 7. Warp icon: a container of 32 threads.

A warp is always 32 threads wide, even if the block has fewer than 32 threads: the unused lanes just sit idle.

Let's compare a block of 32 threads with a block of 33 threads on the same GPU!

32 threads: $32/32 = 1$ warp, all of its lanes are active
33 threads: $\lceil 33/32 \rceil = 2$ warps, the second one has a single active thread

One extra thread doubles the warps, and most of the second warp's lanes are wasted!

15 A Fast Look into the CPU Architecture

We will only look at the architecture of the x86 CPU design.

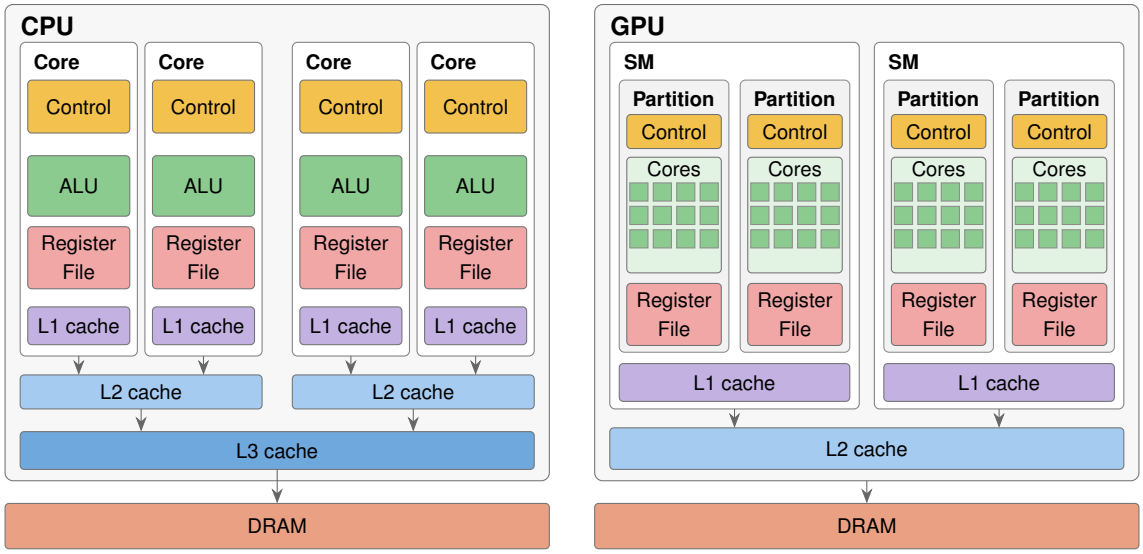


Figure 8. CPU and GPU simplified architectural diagram.

The CPU and the GPU have a lot of similarities, some are real and others are only in the labels!

As you might have noticed, the core counts are very different, and that is because the CPU cores and the GPU cores are not the same thing!

CPU cores, comparable to the GPU's Processing Partitions, as each one has its own Control unit, an ALU which maps to the GPU's cores, and a Register File, which includes the general-purpose registers such as AX, BX, . . . and control registers such as CR0 (bit 0 is used to switch from real mode (16-bit) to protected mode (32-bit)). These cores are complex.

CPU caches, you might have noticed that the CPU has one more cache layer:

- L1 is private to the core and is the smallest in size.
- L2 is usually private to the core, but it can also be shared by a cluster of cores, and is larger in size.
- L3 is shared by all of the cores and is the largest in size.

16 Throughput and Latency

Throughput, a measurement of the number of tasks completed in a given period of time, e.g. the number of triangles rendered within a perfect frame time (0.016 s).

Latency, a measurement of the time taken to complete a given task, e.g. the amount of time taken to render a frame.

17 GPU and CPU Comparison

Silicon partitioning, the GPU prioritizes a larger area of its silicon for the CUDA cores (ALUs) and their pipeline, while the CPU prioritizes the control units and cache.

Arithmetic and Logic Units, the GPU prioritizes many less complex ALUs, while the CPU has a few highly complex ALUs.

Latency hiding, the GPU hides latency by swapping stalled warps, and parallelism also contributes, while the CPU uses its large cache and branch prediction.

Now let's go back to throughput and latency. We have two designs that we can label the CPU and the GPU with: the throughput-oriented design and the latency-oriented design. Let's look further into them.

The CPU has fewer ALUs, so it can do much less work in a given amount of time. That is wrong: because of the way it uses the cache, its delay is very small, which gives it the ability to do much more work sequentially, so we can say it has a latency-oriented design.

The GPU has a fast memory pipeline but is slow at handling many requests, which would lead to low processing power. That is wrong too: it counters this with a massive number of ALUs working in parallel to cover up the latency.

18 Source Material

- Debunking the 100X GPU vs. CPU Myth: An Evaluation of Throughput Computing on CPU and GPU
- CS 5220 - GPU architecture study
- GPU Characteristics - Understanding GPU Architecture
- Nvidia's CUDA Guide
- Basics on Nvidia GPU architecture
- Non-Uniform L2 Cache Latency Across the Streaming Multiprocessors of an NVIDIA L40